Divide and Conquer: Examples

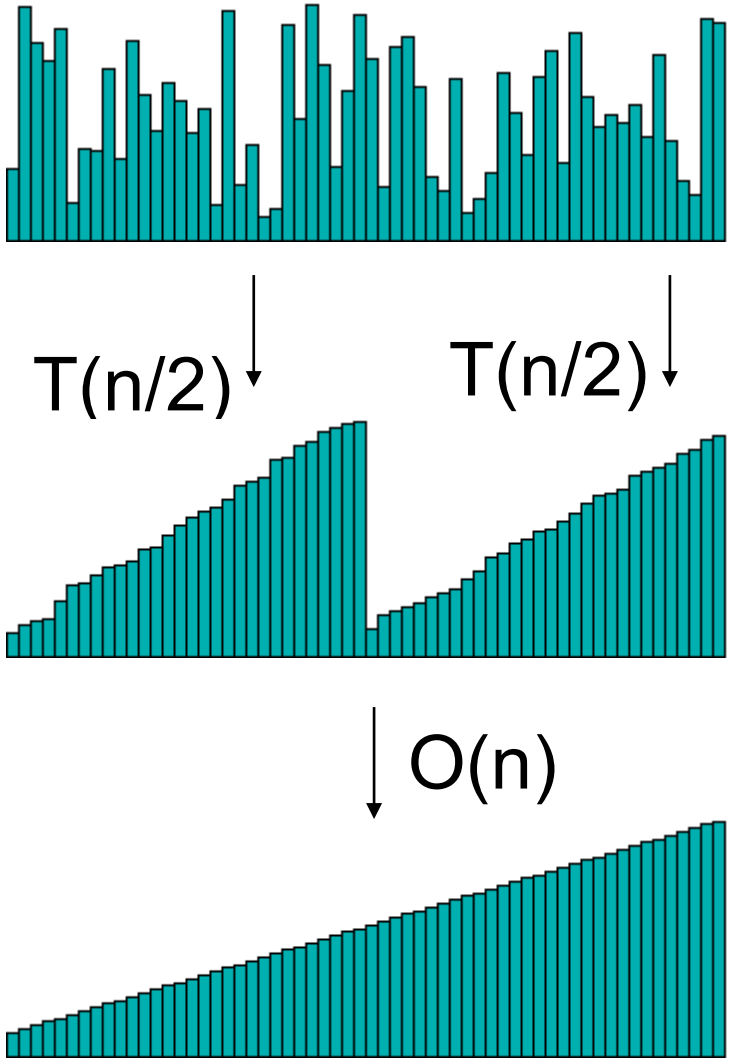
Design and Analysis of Algorithms

Brian C. Dean

School of Computing

Clemson University

Divide and Conquer Examples



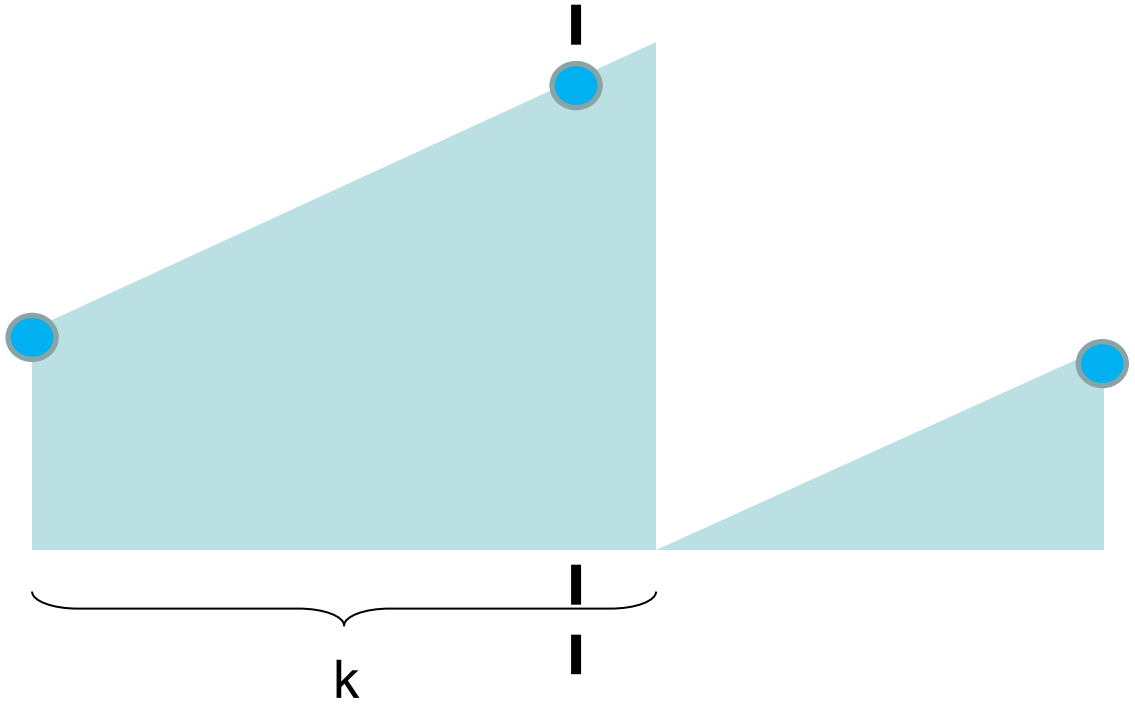
Merge Sort

$T(n) = 2T(n/2) + O(n)$

Solves to $T(n) = O(n \log n)$.

Example: Cyclic Shift Testing

- A sorted array A[1…n] of numbers has been subject to a right cyclic shift by k positions.
- Given the contents of the shifted array, please determine k quickly.



- “Binary search”. T(n) = T(n/2) + O(1) solves to T(n) = O(log n).

Example: Repeated Squaring

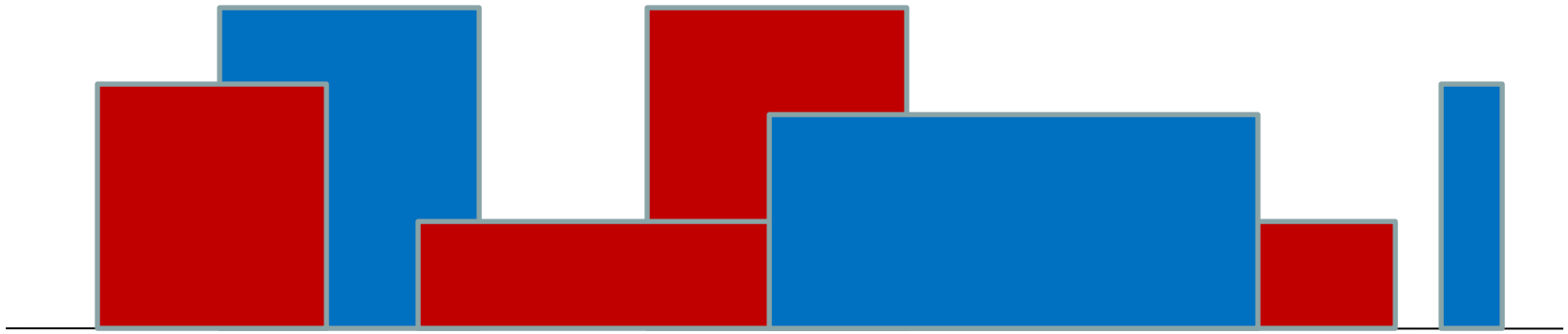
- Calculate $X^{99999999999999999999999999999999999999999999}$.

- $X^N = (X^{N/2})^2$ if N is even.
- $X^N = X * X^{N-1}$ if N is odd.

- Number of multiplications described by T(N) = T(N/2) + O(1), solving to O(log N).

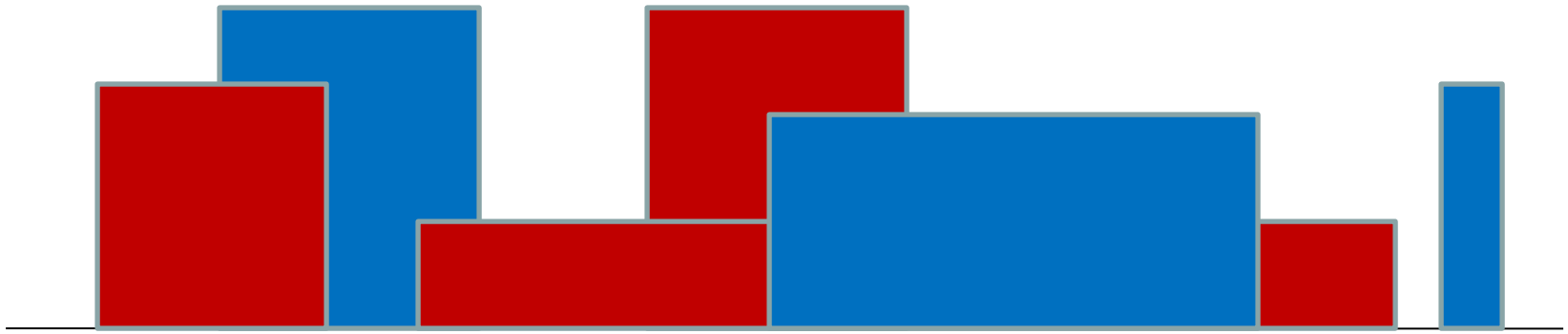
Example: The “Skyline” Problem

- Find the total area of a skyline defined by a union of rectangular buildings with a common base:



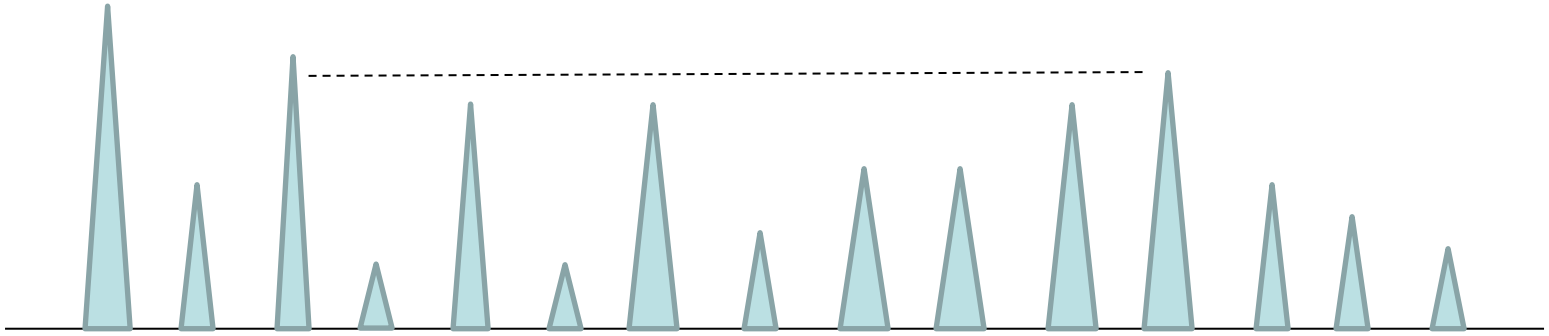
Example: The “Skyline” Problem

- Find the total area of a skyline defined by a union of rectangular buildings with a common base:



- All you need is merge! $T(n) = 2T(n/2) + O(n)$ solves to $O(n \log n)$.

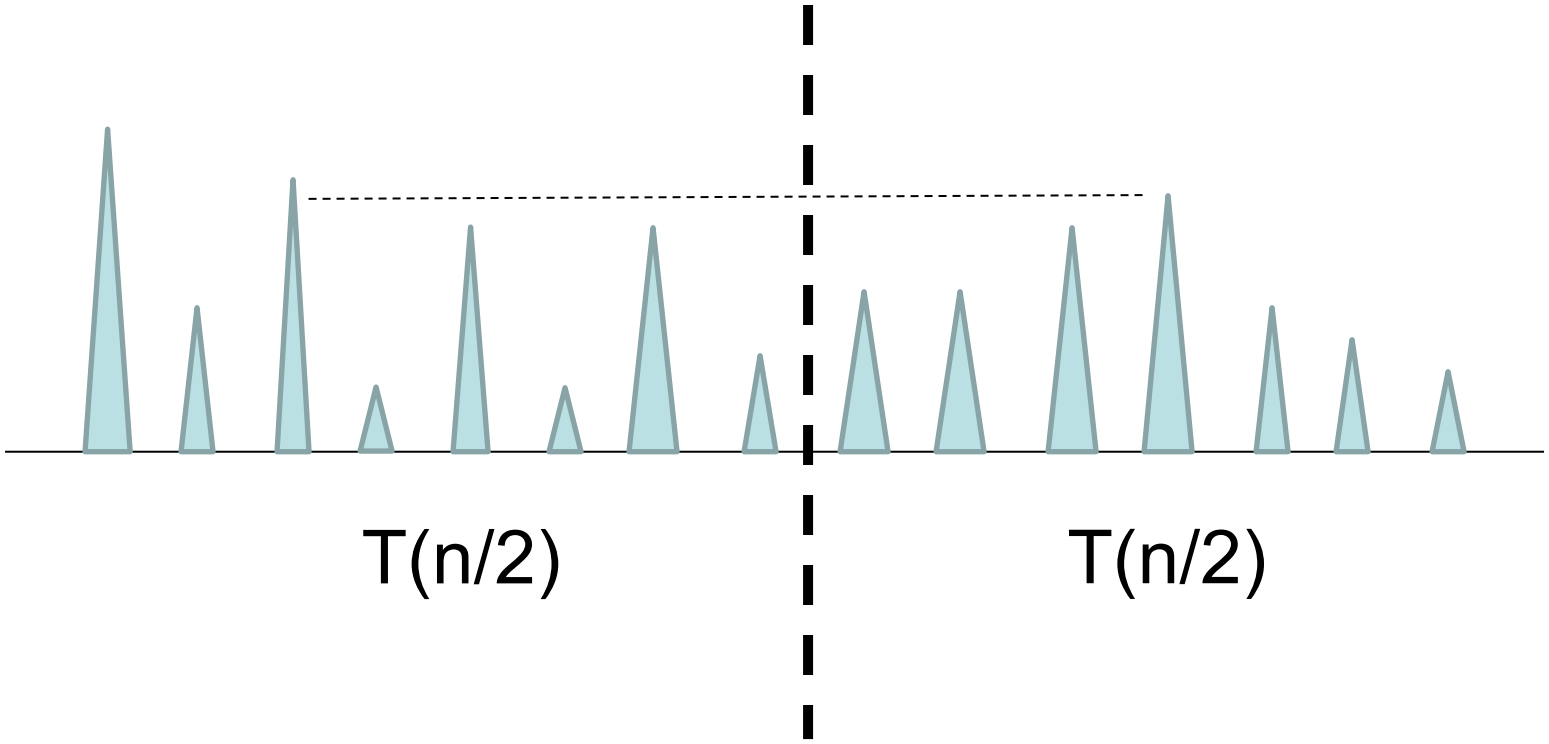
Example: Longest Line of Sight



- Given N distinct heights of individuals standing in a line.
- **Goal:** find the length of the longest line of sight (difference in indices between two people between whom everyone else is shorter).
- Divide and conquer gives us an O(n log n) solution.

  (Recall O(n) also possible from coding discussion in module 1)

Example: Longest Line of Sight

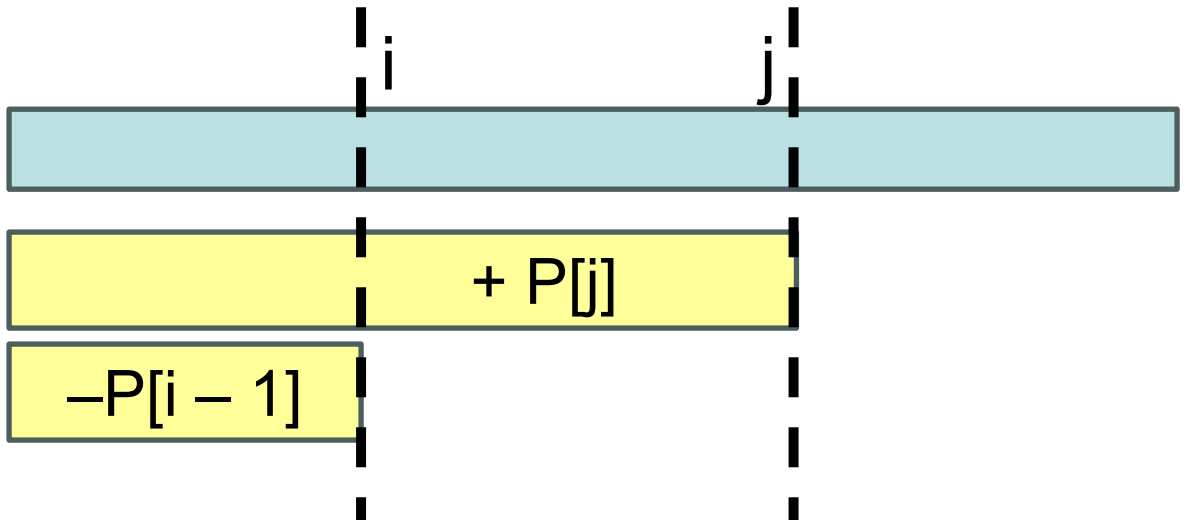


With care, can find longest line of sight crossing the dividing line in O(n) time.

Example: Maximum Sum Subarray

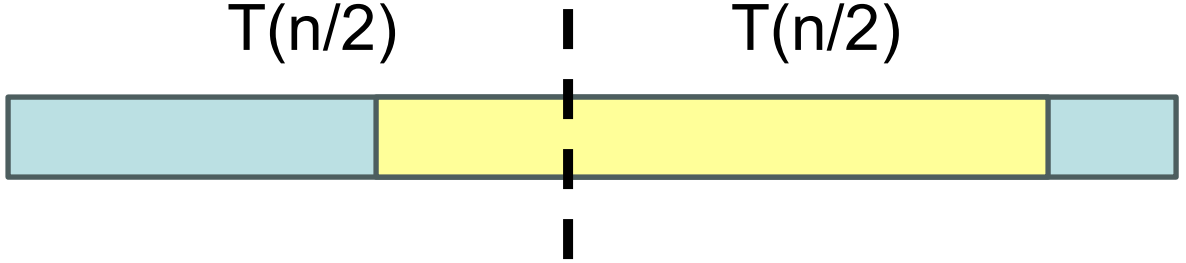
- Given an array A[1…n] of numbers, find a subarray A[i…j] whose elements have maximum sum.

- Trivial approach: spend O(n) time checking all $\binom{n}{2}$ subarrays, for a total of $O(n^3)$.
- Slightly faster: use prefix sums to check each subarray in O(1) time, for a total of $O(n^2)$.



Example: Maximum Sum Subarray

- Given an array A[1…n] of numbers, find a subarray A[i…j] whose elements have maximum sum.

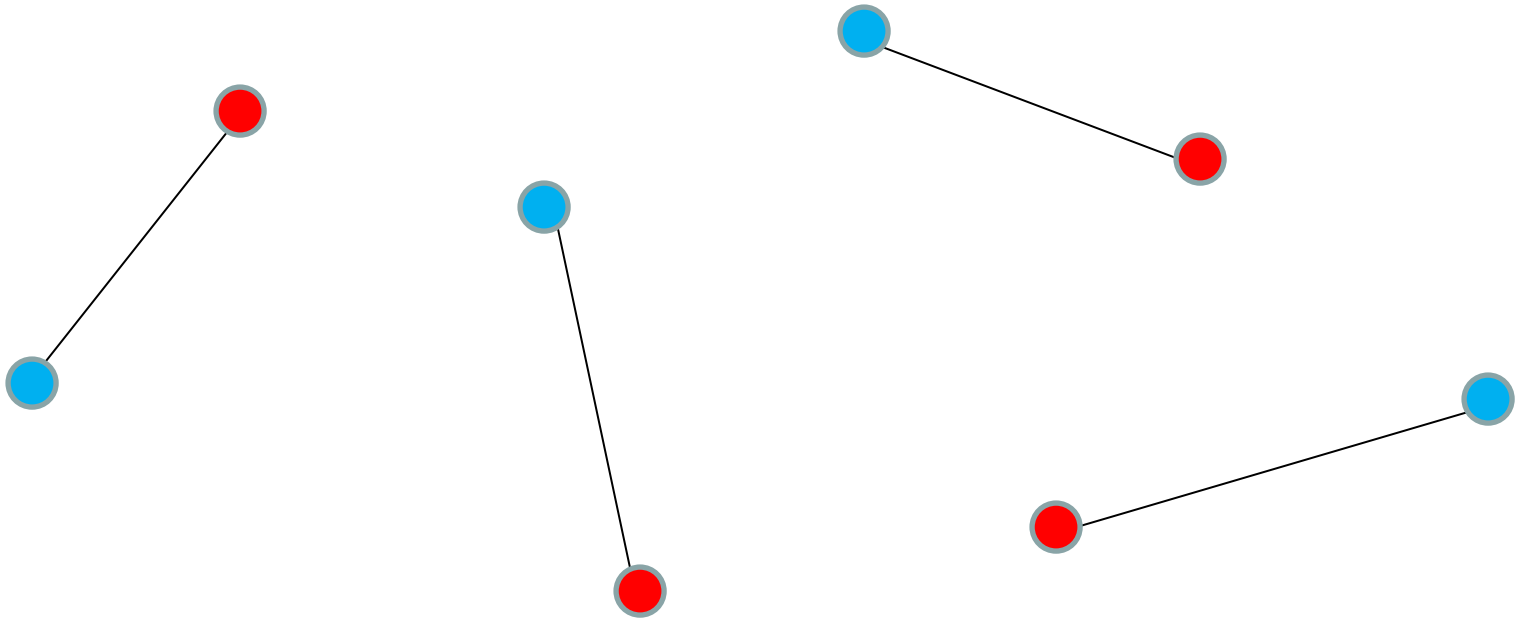


Best suffix sum on left + best prefix sum on right: O(n)

- Better yet: Use divide and conquer -- O(n log n).
- Best: Dynamic programming gives an even simpler O(n) algorithm, which we'll discuss later in the course.

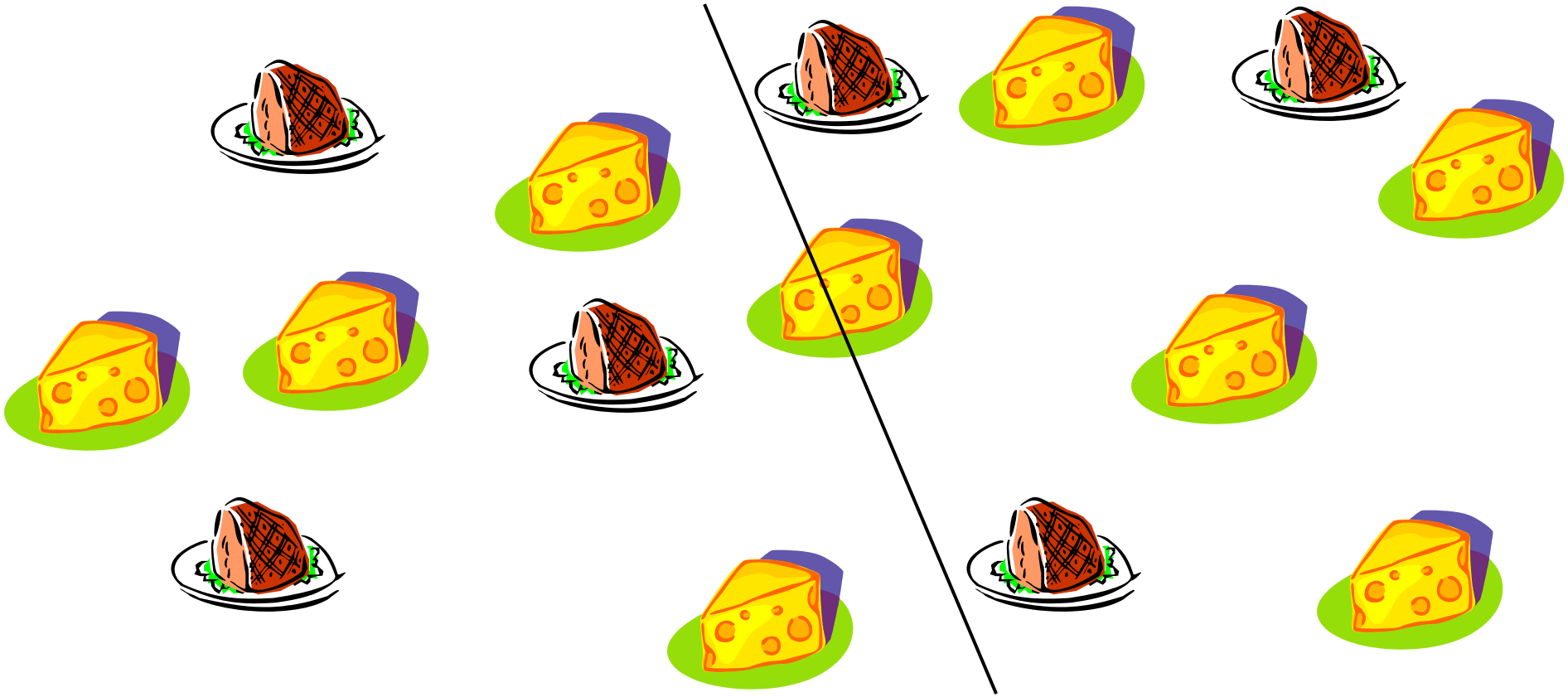
Example: Bi-Chromatic Matching

- Given N red points and N blue points in the plane, match them up with non-crossing line segments…



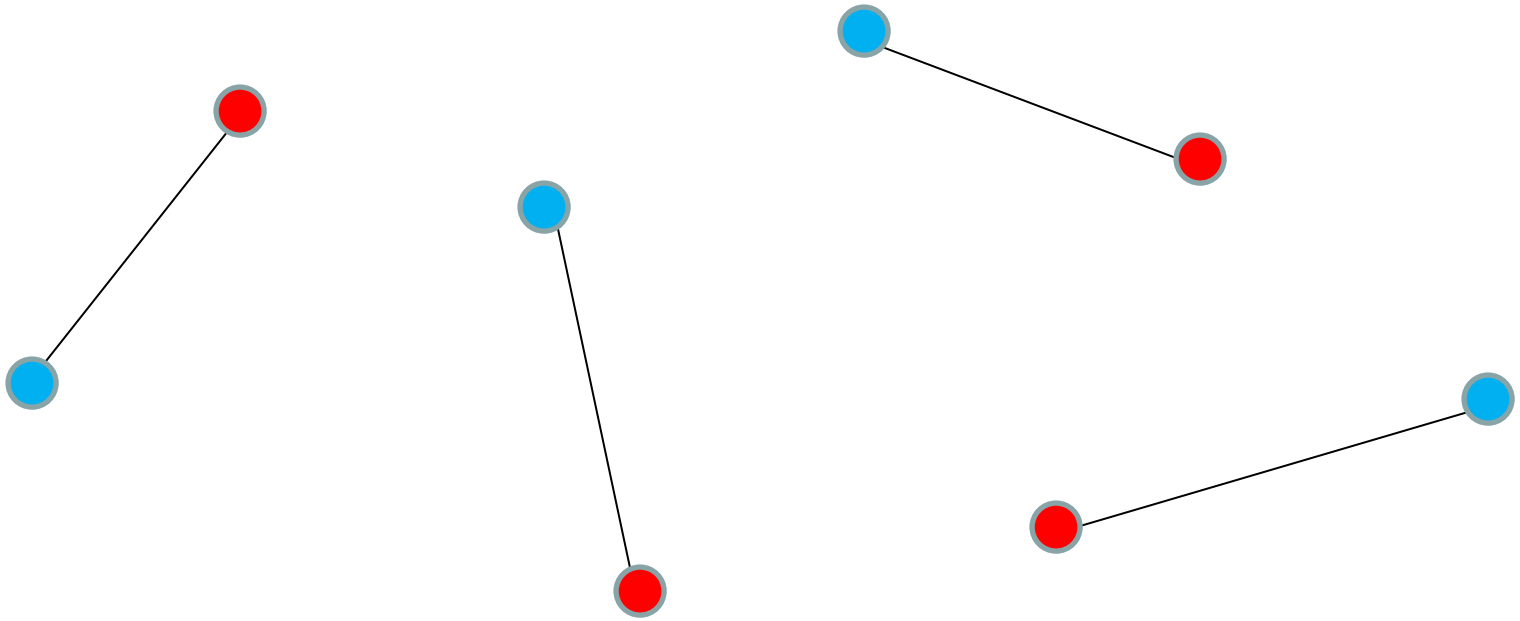
Ham Sandwich Cuts

- It's always possible to find a line in O(n) time that equally subdivides both the ham and the cheese!



Bi-Chromatic Matching

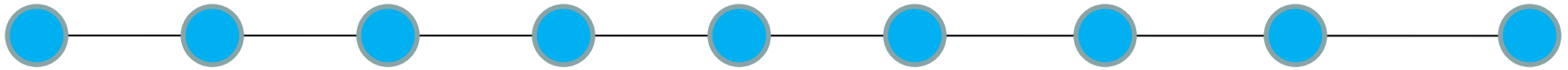
- Given N red points and N blue points in the plane, match them up with non-crossing line segments…



- Easy to solve recursively after partitioning with a ham sandwich cut!

Example: The Firing Squad Problem

- N parallel processors hooked together in a line:

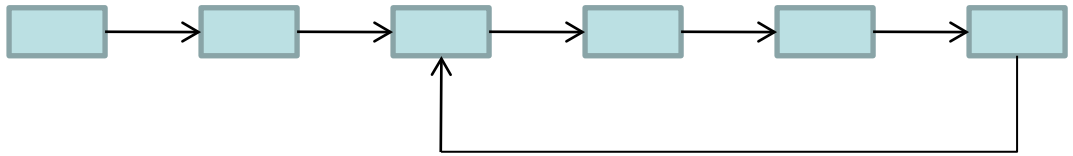


- Each processor doesn't know N, and only has a constant number of bits of memory (so it can't even count to N).
- Processors synchronized to a global clock. In each time step, a processor can:
  - Perform some simple calculation.
  - Exchange messages with its neighbors.
- At some point in time, give leftmost processor a "ready!" message.
- Sometime in the future, we want all the processors to enter the same state "fire!" all in the same time step.

Aside: Linked List Ending in a Loop

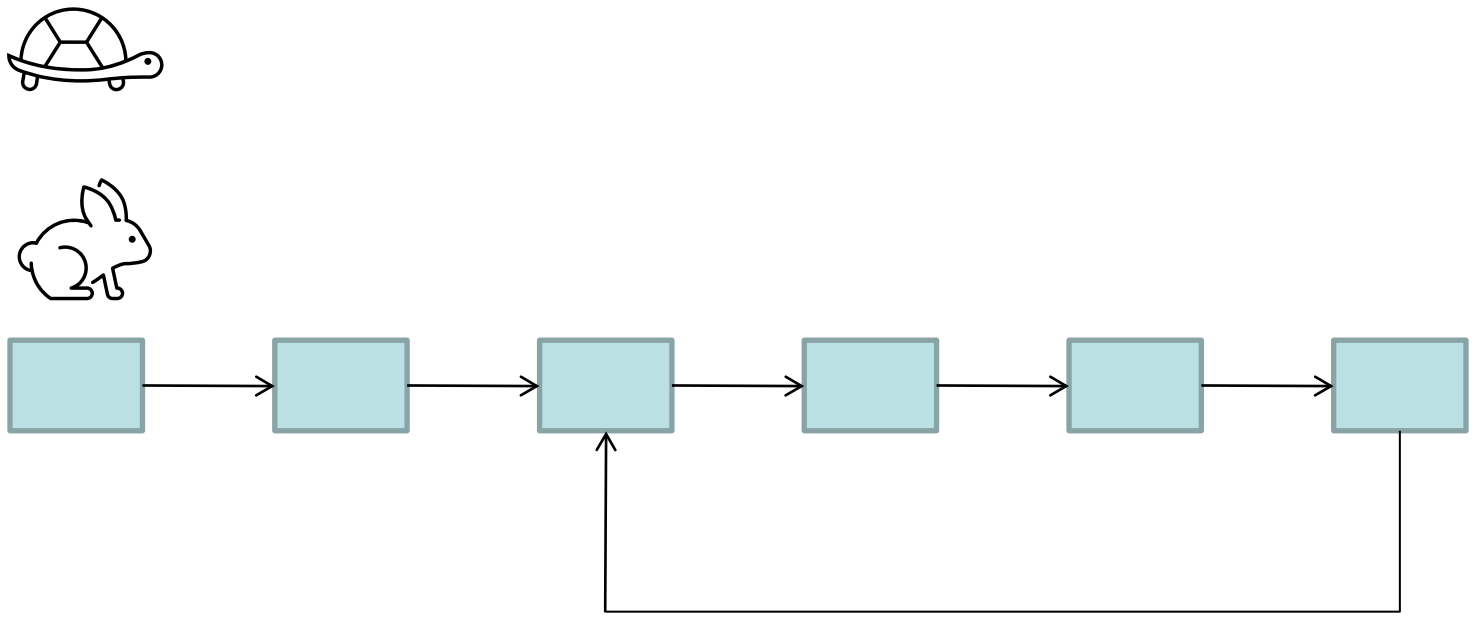
- Given a pointer to the beginning of a linked list.
- It either ends by pointing to NULL:

or it ends by pointing back into itself, forming a loop:

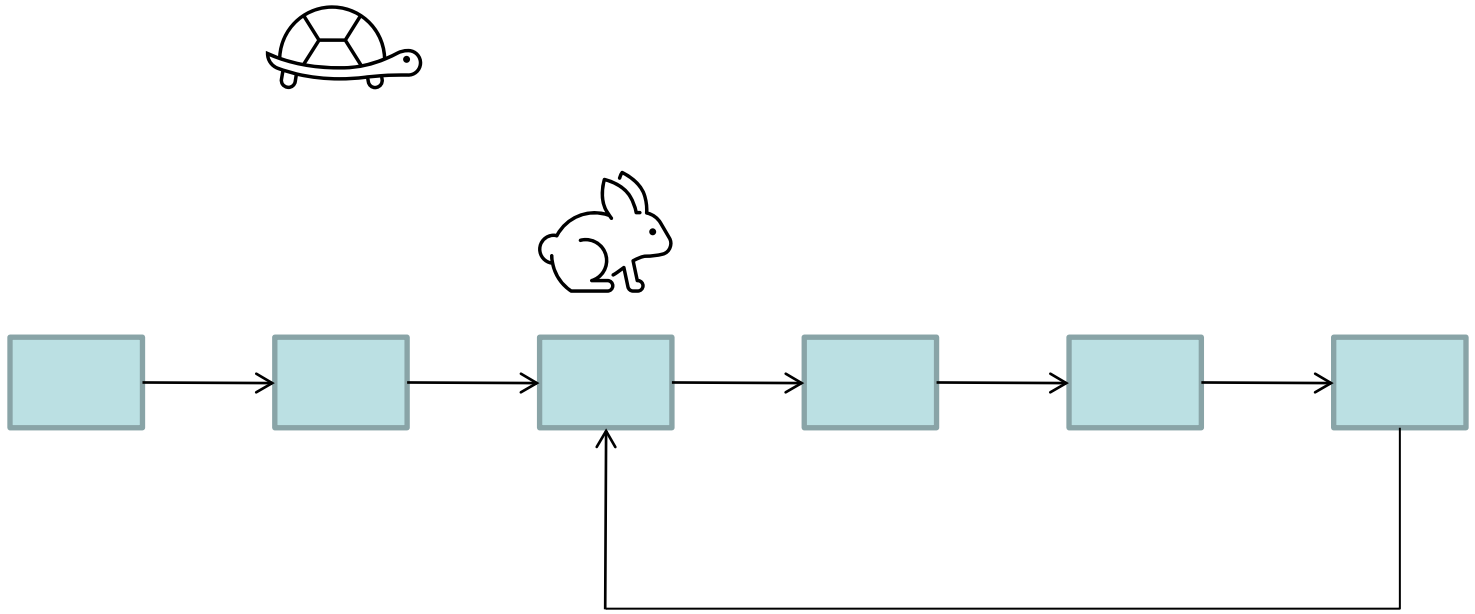


- **Goal:** Determine which of these two cases is occurring.
- **To make things interesting:** You aren't allowed to modify the list, or to use substantial amounts of extra memory…

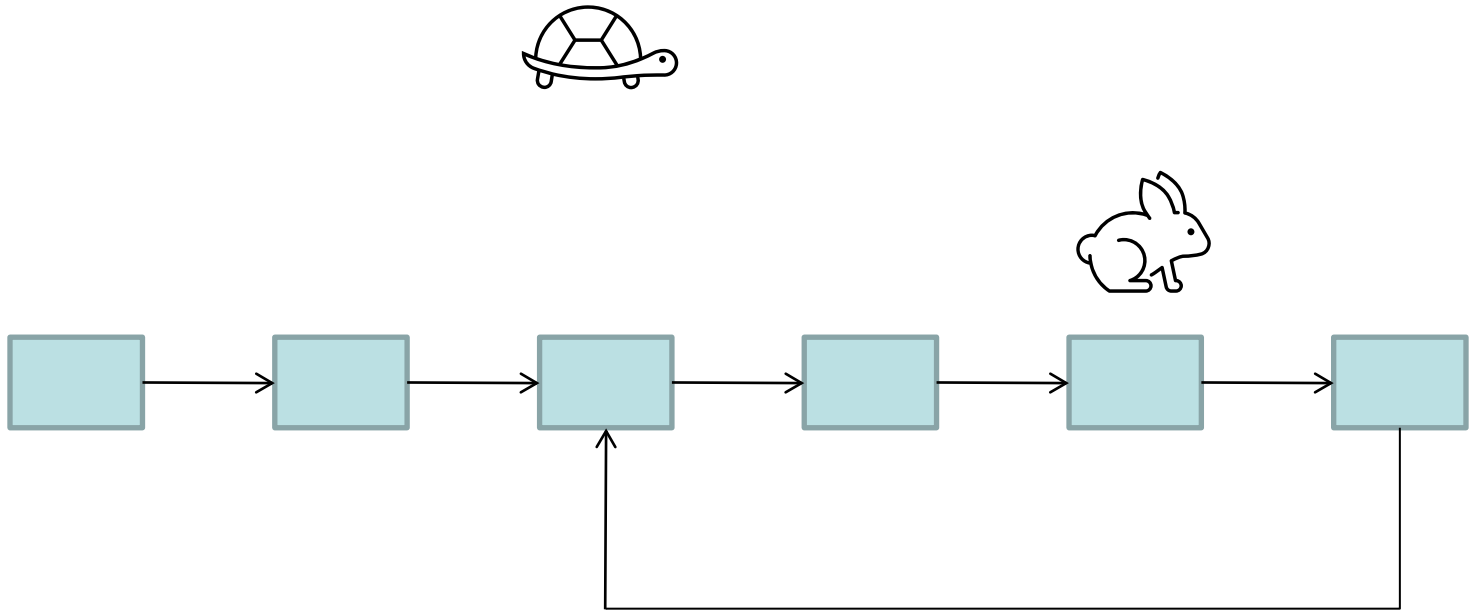
Slow / Fast Pointer Tricks



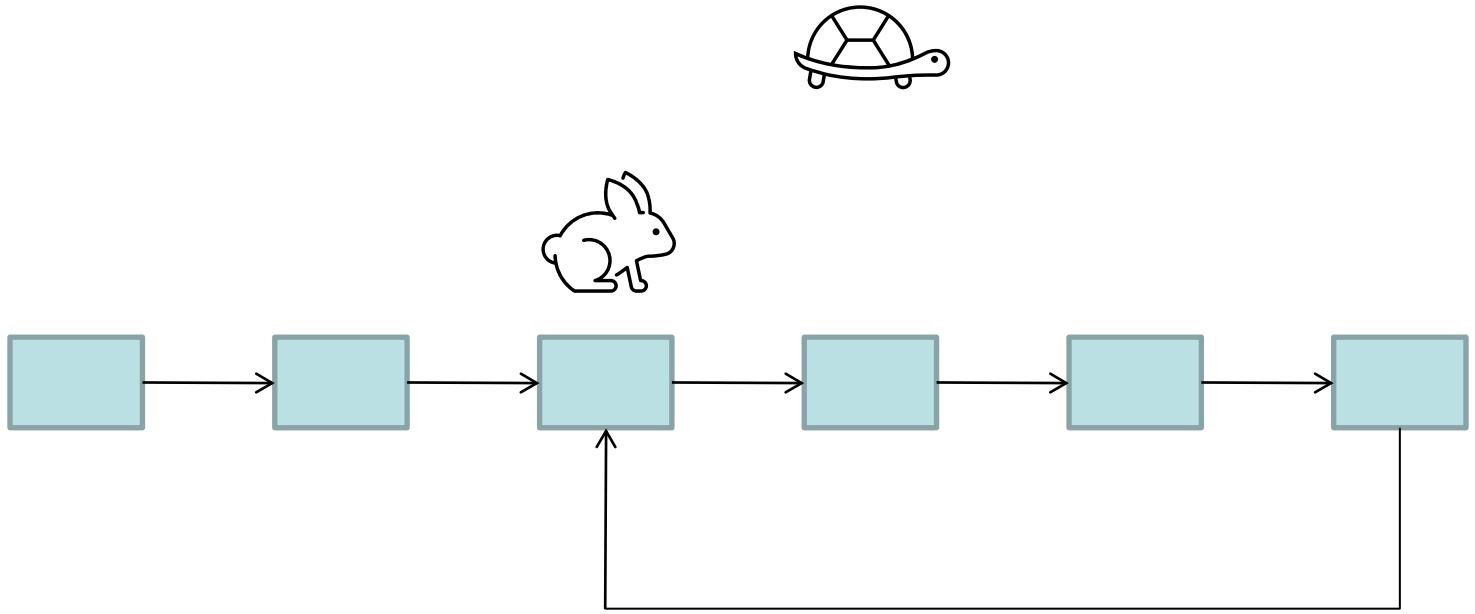
Slow / Fast Pointer Tricks



Slow / Fast Pointer Tricks

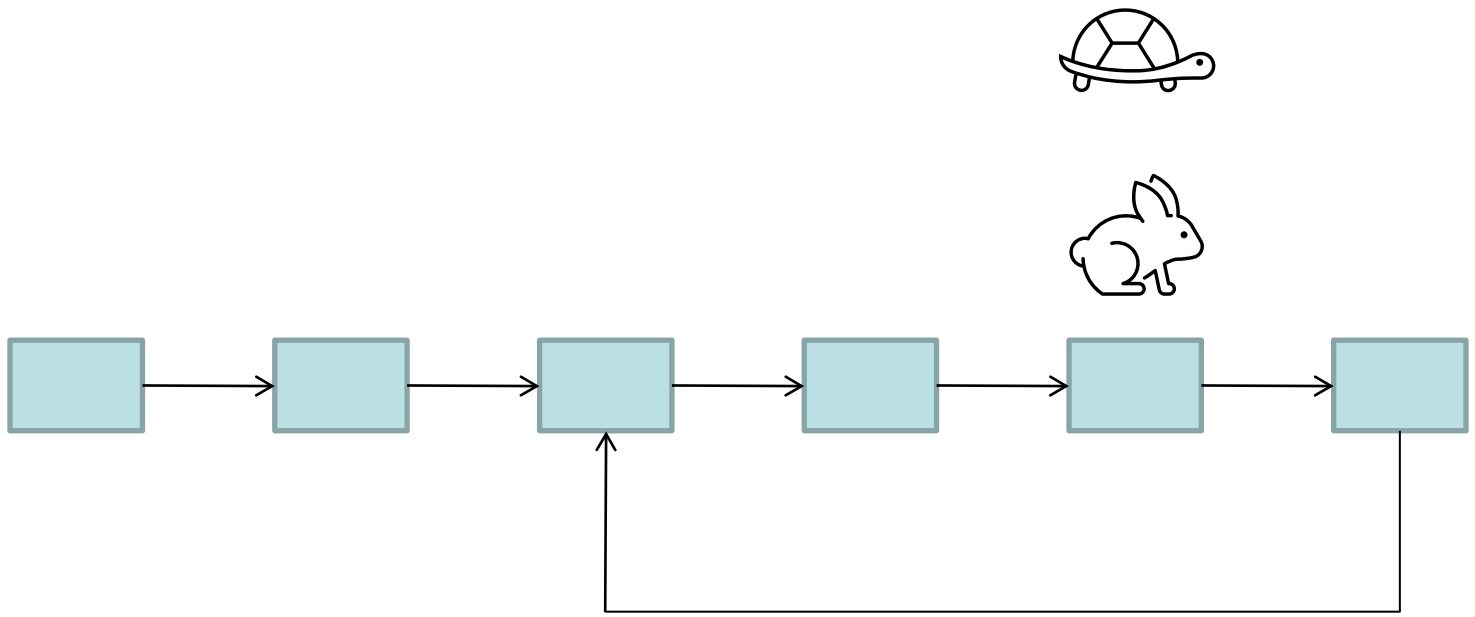


Slow / Fast Pointer Tricks

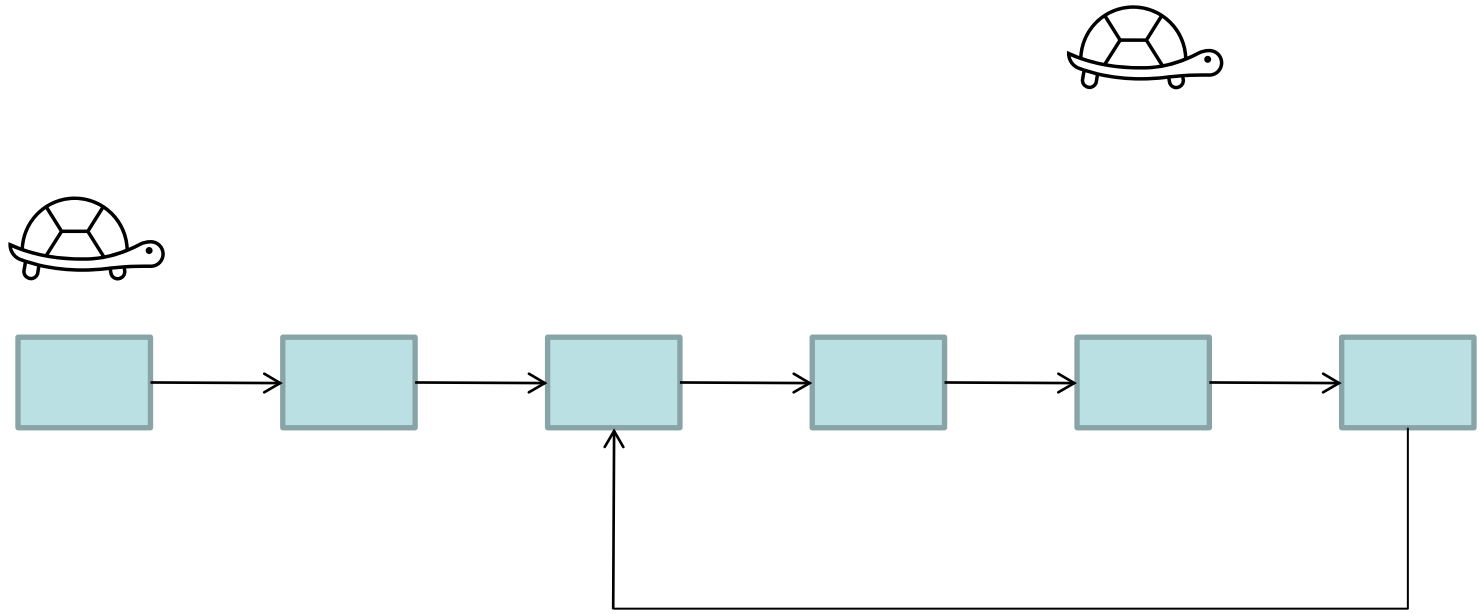


Slow / Fast Pointer Tricks

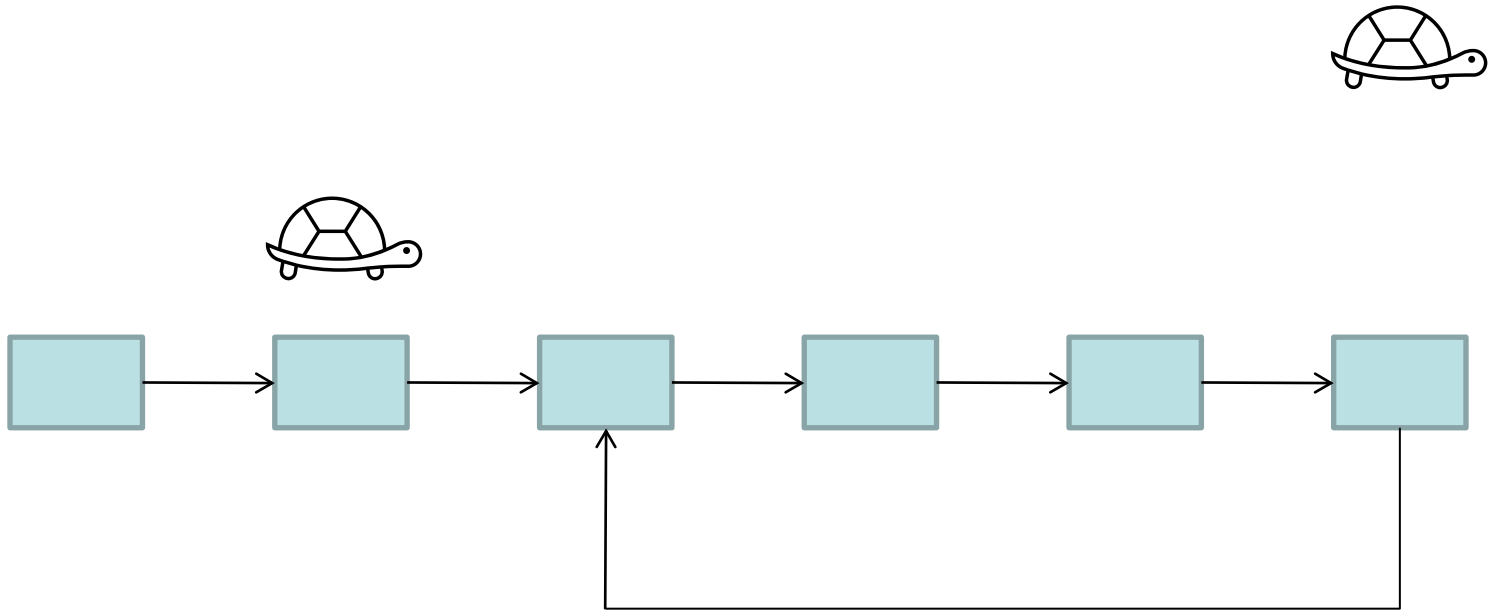
After T = O(n) steps



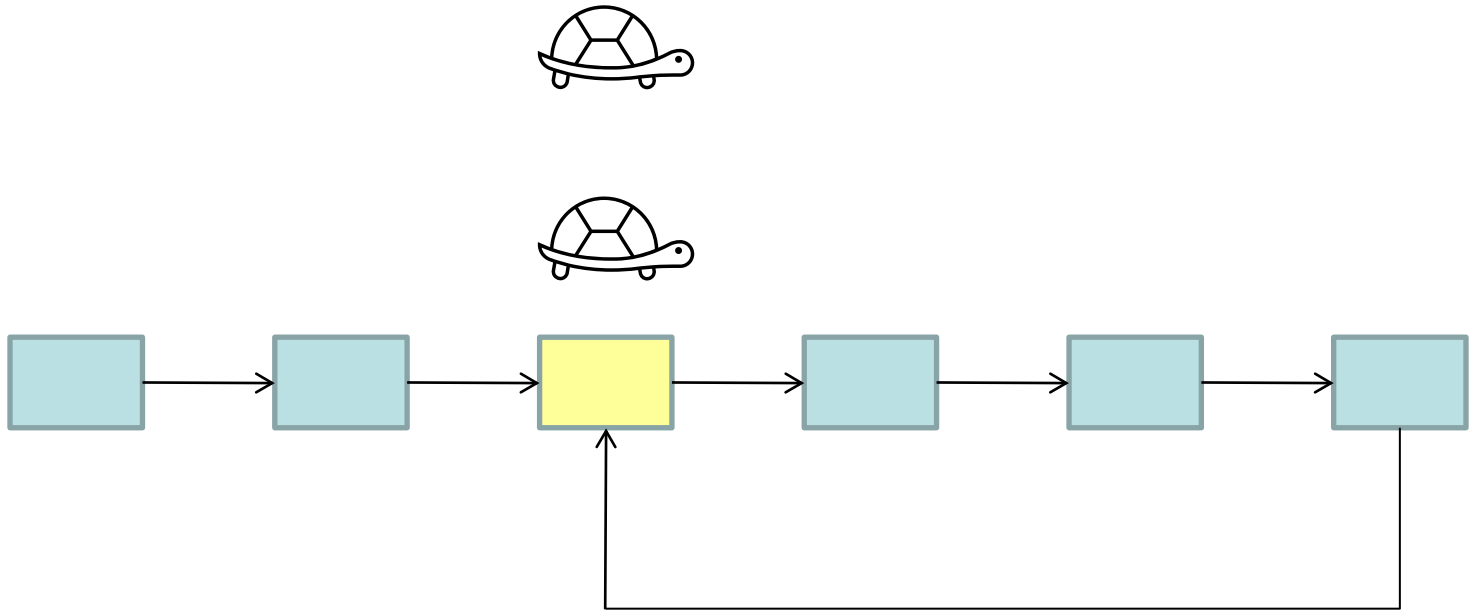
Slow / Fast Pointer Tricks



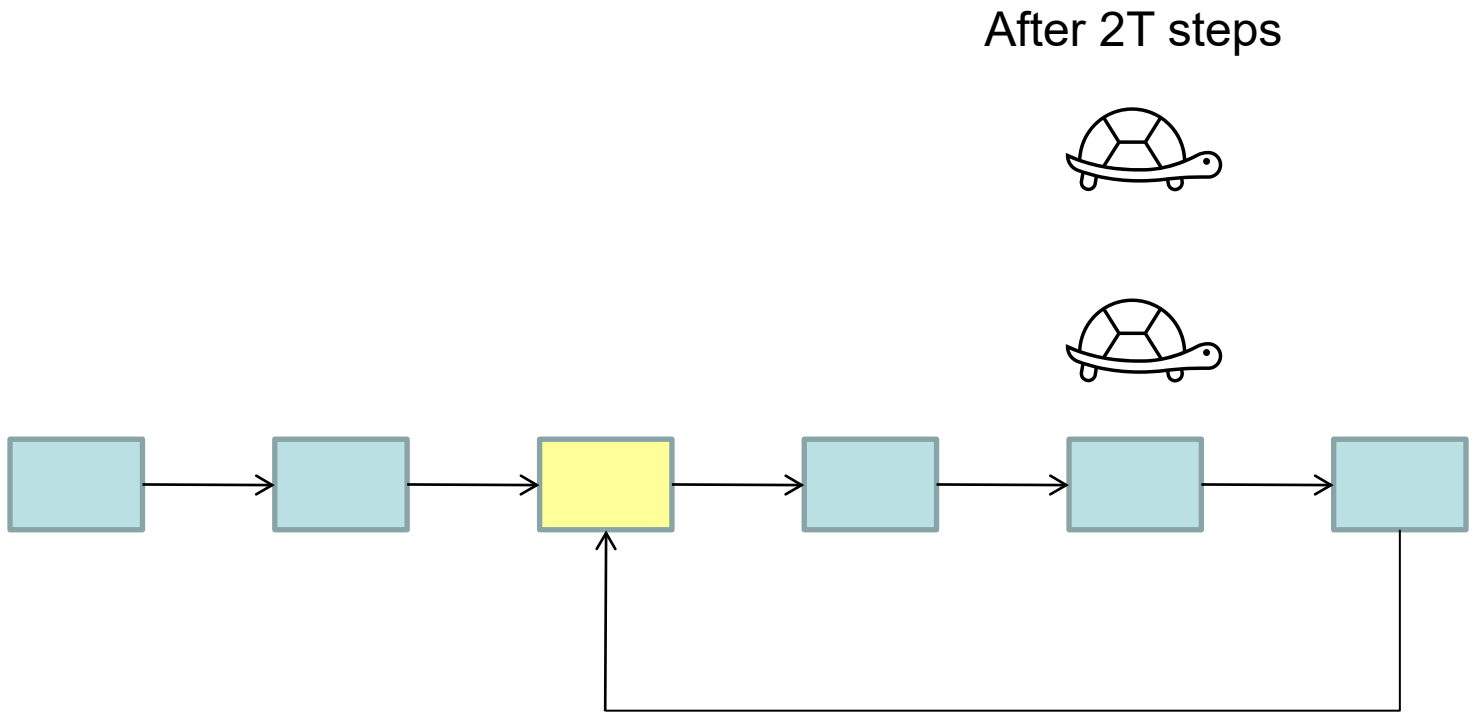
Slow / Fast Pointer Tricks



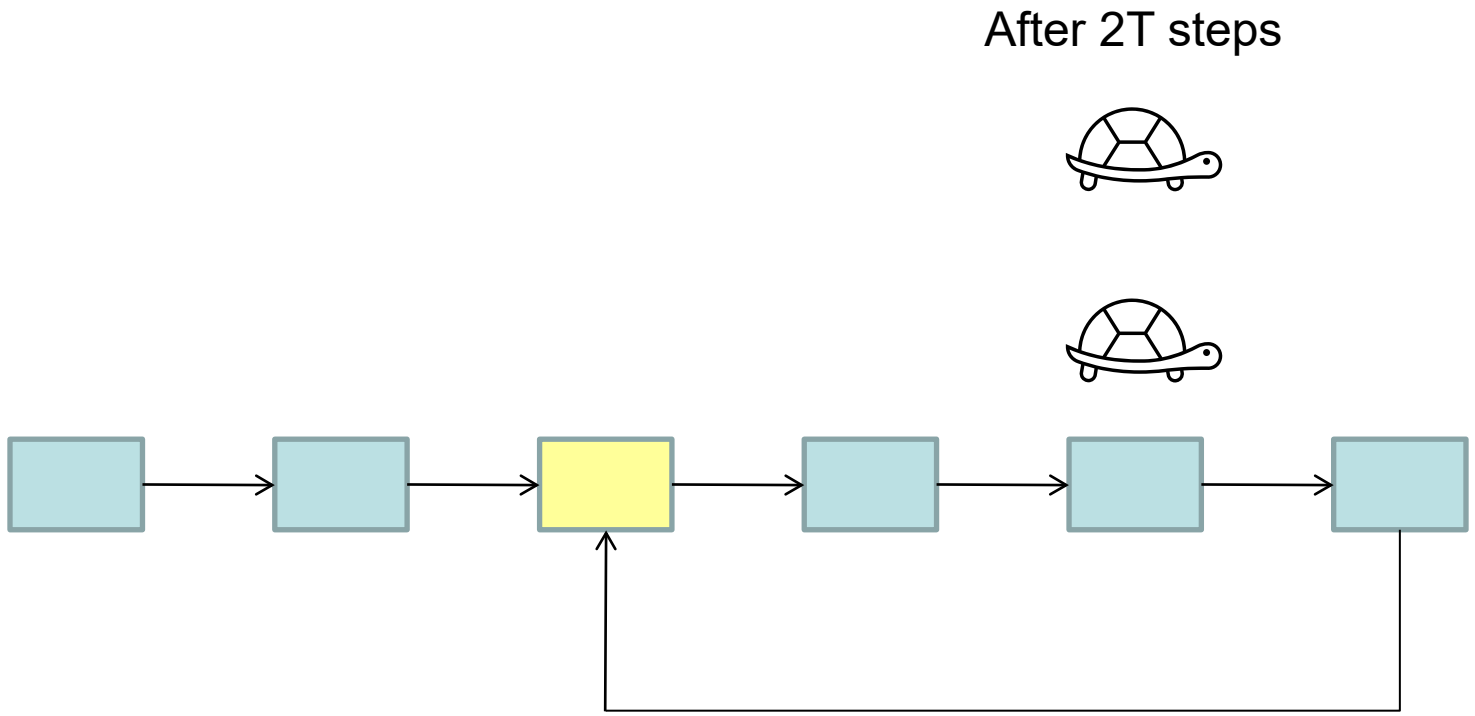
Slow / Fast Pointer Tricks



Slow / Fast Pointer Tricks



Slow / Fast Pointer Tricks

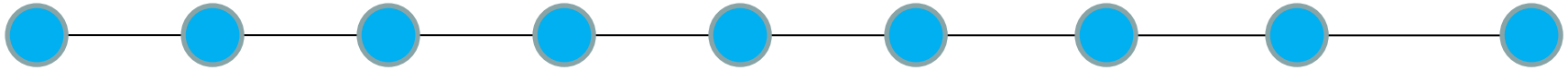


Pollard's "rho" heuristic helps with a surprising number of problems (e.g., it can even speed up factoring!)

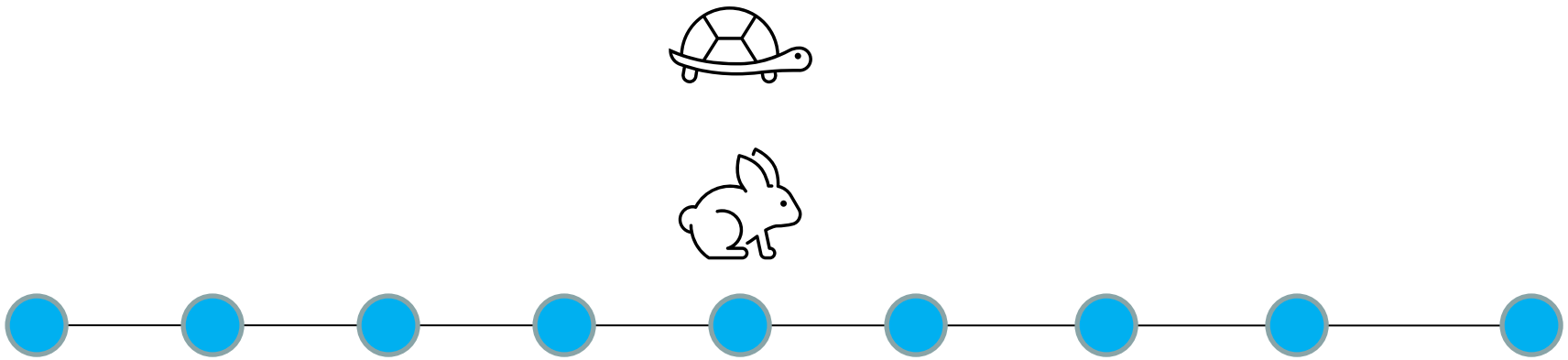
Back to The Firing Squad Problem

Speed = 1/3 

Speed = 1 



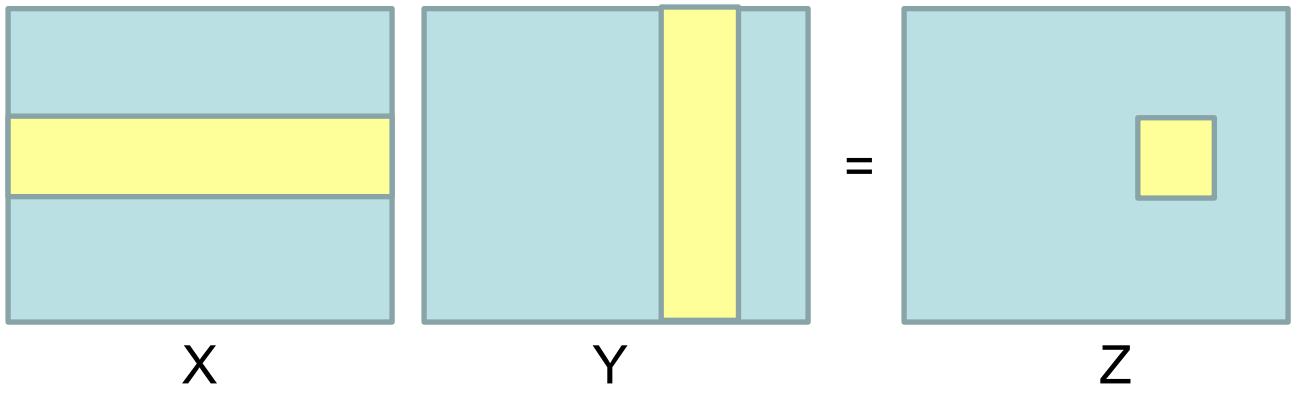
Back to The Firing Squad Problem



- Slow and fast points meet in middle after O(n) steps.
- Then repeat process recursively on left and right.
- T(n) = O(n) + T(n/2), solving to O(n).
- Note: recurrence isn't T(n) = O(n) + 2T(n/2), since in parallel.

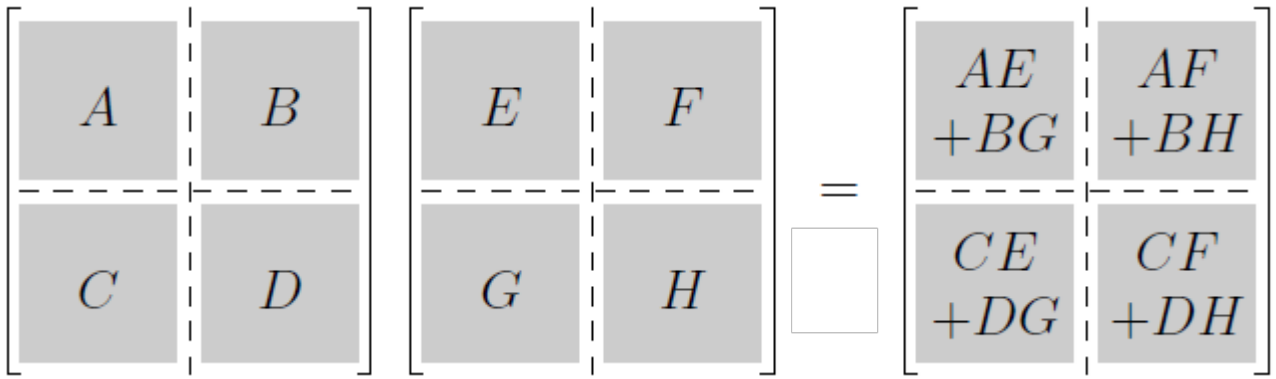
Example: Matrix Multiplication

- Multiply two N x N matrices…



$$Z_{ij} = \sum_k X_{ik} Y_{kj}$$

Θ(N) per element of Z,
So Θ(N^3) total



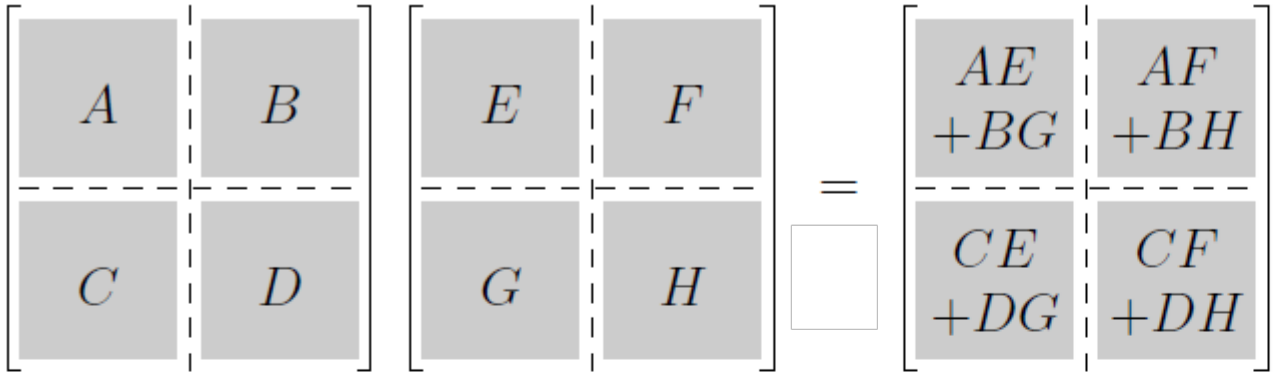
<u>Multiplication by blocks:</u>

- 8 recursive multiplications on N/2 x N/2 matrices: 8T(N/2)
- 4 additions on N/2 x N/2 matrices: Θ(N^2)

T(N) = 8T(N/2) + Θ(N^2)
Solves to T(N) = Θ(N^3) ☹

Strassen's Algorithm

- Multiply two N x N matrices…



$$
\begin{aligned}
AE + BG &= P_1 + P_4 - P_5 + P_7 \\
AF + BH &= P_3 + P_5 \\
CE + DG &= P_2 + P_4 \\
CF + DH &= P_1 - P_2 + P_3 + P_6
\end{aligned}
$$

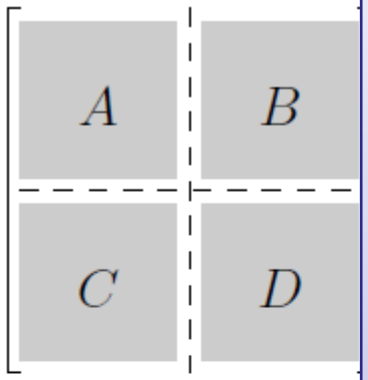
where...

$$
\begin{aligned}
P_1 &= (A + D)(E + H) \\
P_2 &= (C + D)E \\
P_3 &= A(F - H) \\
P_4 &= D(G - E) \\
P_5 &= (A + B)H \\
P_6 &= (C - A)(E + F) \\
P_7 &= (B - D)(G + H)
\end{aligned}
$$

- Only 7 recursive multiplies on N/2 x N/2 matrices: 7T(N/2)
- 18 additions on N/2 x N/2 matrices: $\Theta(N^2)$
- New recurrence: $T(N) = 7T(N/2) + \Theta(N^2)$
- Solves to $T(N) = \Theta(N^{\log_2 7}) \approx \Theta(N^{2.81})$

Strassen's Algorithm

- Multiply two N x N matrices…



$+ P_4 - P_5 + P_7$
$+ P_5$
$+ P_4$
$- P_2 + P_3 + P_6$

$A + D)(E + H)$
$C + D)E$
$(F - H)$
$(G - E)$
$A + B)H$
$C - A)(E + F)$
$B - D)(G + H)$

Matrix Multiplication…
Best known lower bound: $\Omega(n^2)$ (trivial)

Strassen 1969: $O(n^{2.81})$
Coppersmith and Winograd 1990: $O(n^{2.376})$
(However, it's quite complicated and not very practical)
Stothers 2011: $O(n^{2.374})$
Williams 2012: $O(n^{2.3729})$
Le Gall 2014: $O(n^{2.37287})$

- Only 7 rec
  N/2 x N/2
- 18 additio
- New recur
- Solves to $T(N) = \Theta(N^{\log_2 7}) \approx \Theta(N^{2.81})$